

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Криптографические методы обеспечения информационной безопасности»

КУРСОВАЯ РАБОТА

«Проектирование и реализация защищенного шлюза на базе Raspberry Pi 4 с выводом трафика через удаленный сервер в Нидерландах (VPN-over-Wi-Fi Router).»

Выполнил:

Бардышев Артём Антонович,
студент группы N3346



(подпись)

Проверил:

Калабишка Михаил Михайлович,
младший научный сотрудник, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

Введение	3
1 Теоретические основы построения защищенного VPN-шлюза	5
1.1 Модель угроз и требования к системе	5
1.2 Принципы VPN-туннелирования	5
1.3 Криптографическая модель WireGuard	5
1.4 Теория сегментации и маршрутизации	6
2 Проектирование архитектуры решения	7
2.1 Функциональные роли узлов	7
2.2 Обоснование выбора компонентов	7
2.3 Адресный план и политика доступа	7
3 Практическая реализация	8
3.1 Конфигурация NL-сервера	8
3.2 Конфигурация Raspberry Pi как AP	8
3.3 Правила маршрутизации и предотвращение утечек	8
3.4 Автоматизация запуска	8
4 Криптографические механизмы и стойкость	10
4.1 WireGuard	10
4.2 Wi-Fi сегмент (WPA2-PSK)	10
4.3 Криптографическая эффективность	10
4.4 Эксплуатационная эффективность	11
4.5 Анализ рисков и меры снижения	12
5 Проблемы, выявленные на практике, и их решение	13
5.1 Ошибка запуска dnsmasq: unknown interface ap0	13
5.2 SSID виден, но клиенты периодически не подключались после reboot	13
5.3 Проблемы маршрутизации при первичной отладке туннеля	13
5.4 Существование с уже работающими сервисами на NL	14
Заключение	15
Список использованных источников	16

ВВЕДЕНИЕ

В условиях сетевых ограничений и необходимости защиты данных при передаче по недоверенным сетям актуальной задачей является создание автономного и безопасного узла доступа в интернет. Цель данной работы — реализовать портативный сетевой шлюз, который после подачи питания автоматически поднимает Wi-Fi сеть и направляет весь пользовательский трафик через защищенный VPN-туннель на удаленный сервер в Нидерландах.

Практическая ценность проекта состоит в том, что решение обеспечивает:

- конфиденциальность пользовательского трафика в публичных и домашних сетях;
- единый защищенный выход в интернет для нескольких клиентских устройств;
- автономный режим работы "из коробки" (без ручной настройки после перезагрузки);
- разделение ролей: Raspberry Pi как клиентский VPN-шлюз, удаленный сервер как точка egress.

Цель работы – развернуть отказоустойчивую и криптографически защищенную сетевую инфраструктуру "Raspberry Pi 4 -> WireGuard -> NL-сервер -> Интернет", обеспечивающую автоматический запуск и безопасную маршрутизацию трафика.

Задачи:

1. Настроить сервер в Нидерландах как WireGuard endpoint и NAT-шлюз.
2. Настроить Raspberry Pi 4 как Wi-Fi точку доступа (AP) с DHCP/DNS.
3. Реализовать туннелирование клиентского трафика через WireGuard.
4. Внедрить механизмы предотвращения утечек (killswitch-поведение на уровне маршрутизации).
5. Обеспечить автозапуск всех сервисов после cold boot.
6. Провести проверку функциональности и оценку криптографической стойкости выбранных алгоритмов.

Объект и предмет исследования

- Объект: процесс защищенной передачи пользовательского сетевого трафика через VPN-шлюз.
- Предмет: практическое применение современных криптографических примитивов в WireGuard и WPA2-PSK для обеспечения конфиденциальности, целостности и подлинности данных.

Логическая схема - клиентское устройство (смартфон/ноутбук) подключается к Wi-Fi сети, раздаваемой Raspberry Pi. Raspberry Pi выполняет функции L3-шлюза, DHCP/DNS сервера и WireGuard клиента. Дальнейшая передача выполняется по зашифрованному туннелю на NL-сервер, где осуществляется NAT и выход в интернет.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПОСТРОЕНИЯ ЗАЩИЩЕННОГО VPN-ШЛЮЗА

1.1 Модель угроз и требования к системе

В рамках работы рассматривается сценарий, в котором пользователь подключается к недоверенной или частично доверенной сети и должен обеспечить:

1. конфиденциальность трафика;
2. защиту от подмены и модификации пакетов;
3. отсутствие "утечки" трафика в обход VPN;
4. автономность (работа после подачи питания без ручных действий).

К основным угрозам относятся перехват трафика на участке "клиент -> точка доступа", анализ трафика провайдером доступа и ошибки конфигурации, приводящие к выходу трафика в интернет вне туннеля.

1.2 Принципы VPN-туннелирования

VPN-туннель создает логический защищенный канал поверх публичной сети. В данной работе туннель используется между Raspberry Pi и NL-сервером. Все клиентские пакеты сначала агрегируются на Raspberry Pi, затем инкапсулируются и шифруются WireGuard, после чего передаются на удаленный сервер.

Такой подход обеспечивает вынесение точки выхода (egress) в другую юрисдикцию и скрывает локальный IP клиентов за публичным IP NL-сервера.

1.3 Криптографическая модель WireGuard

WireGuard использует фиксированный набор современных примитивов:

1. Curve25519 — обмен ключами (ECDH);
2. ChaCha20 — шифрование;
3. Poly1305 — контроль целостности и аутентификация;
4. BLAKE2s/HKDF — хеширование и деривация ключей.

Фиксированный набор алгоритмов снижает вероятность ошибок администратора при выборе параметров шифрования и устраняет риск использования устаревших cipher suites.

1.4 Теория сегментации и маршрутизации

Для корректной работы системы применено разделение сетевых зон:

1. клиентская зона Wi-Fi (`192.168.77.0/24`);
2. транспортная VPN-зона (`10.66.0.0/24`);
3. внешняя зона интернета (публичный интерфейс NL).

Сегментация позволяет формально задать единственный разрешенный путь для пользовательского трафика: `ap0 -> wg0 -> NL -> internet`.

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ РЕШЕНИЯ

2.1 Функциональные роли узлов

1. Raspberry Pi 4:

1. Wi-Fi точка доступа;
2. DHCP/DNS сервер;
3. VPN-клиент WireGuard;
4. шлюз и фильтр трафика.

2. NL сервер:

1. VPN endpoint;
2. NAT-шлюз в интернет;
3. удаленная точка выхода пользовательского трафика.

2.2 Обоснование выбора компонентов

1. WireGuard выбран из-за современного криптографического набора и высокой производительности на ARM.
2. hostapd выбран как стандартный и устойчивый механизм поднятия AP в Linux.
3. dnsmasq выбран как легковесный DHCP/DNS-комбайн для встроенных платформ.
4. systemd выбран для детерминированного автозапуска и контроля зависимостей между сервисами.

2.3 Адресный план и политика доступа

1. NL (`wg0`) — `10.66.0.1/24`.
2. RPi (`wg0`) — `10.66.0.3/24`.
3. AP на RPi (`ap0`) — `192.168.77.1/24`.
4. DHCP pool — `192.168.77.50-192.168.77.200`.
5. Разрешенный форвардинг — только `ap0 -> wg0`.
6. Обратный трафик — только `RELATED,ESTABLISHED`.

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

3.1 Конфигурация NL-сервера

На NL выполнены следующие действия:

1. поднят интерфейс `wg0` и добавлены peers;
2. включена пересылка пакетов `net.ipv4.ip\_forward=1`;
3. добавлены правила `FORWARD wg0<->eth0`;
4. настроен NAT (`SNAT/MASQUERADE`) для VPN-подсети.

Для чего это сделано: чтобы сервер мог выполнять функции выхода в интернет для всех клиентов, приходящих из туннеля.

3.2 Конфигурация Raspberry Pi как AP

На Raspberry Pi реализованы:

1. отдельный AP-интерфейс `ap0`;
2. подъем Wi-Fi через `hostapd` (SSID `Pi-NL-Net`);
3. выдача адресов и DNS через `dnsmasq`;
4. подключение к NL через `wg0`.

Для чего это сделано: чтобы пользовательское устройство могло получить сеть в формате "подключился к Wi-Fi -> интернет уже защищен".

3.3 Правила маршрутизации и предотвращение утечек

Реализованы фильтры:

1. `FORWARD ap0 -> wg0 ACCEPT`;
2. `FORWARD wg0 -> ap0 RELATED,ESTABLISHED ACCEPT`;
3. `FORWARD ap0 !-> wg0 DROP`;
4. `POSTROUTING 192.168.77.0/24 -> wg0 MASQUERADE`.

Почему это важно: даже при ошибке маршрутов или временном сбое туннеля клиентский трафик не уходит в обход VPN.

3.4 Автоматизация запуска

Для режима "из коробки" включены сервисы:

1. `ap0.service`;
2. `hostapd.service`;

3. ``dnsmasq.service``;
4. ``wg-quick@wg0.service``;
5. ``netfilter-persistent``.

Также добавлены зависимости запуска ``hostapd/dnsmasq`` после создания ``ap0``, что устраняет гонки и нестабильность при cold boot.

4 КРИПТОГРАФИЧЕСКИЕ МЕХАНИЗМЫ И СТОЙКОСТЬ

4.1 WireGuard

WireGuard применяет современный фиксированный криптографический набор (без устаревших алгоритмов):

- Обмен ключами: Curve25519 (ECDH)
- Шифрование: ChaCha20
- Контроль целостности/аутентификация: Poly1305
- Хеширование/ключевая деривация: BLAKE2s, HKDF
- Счетчики/nonce: защищают от повторов (replay)

Оценка криптостойкости

- Curve25519 обеспечивает высокий уровень стойкости при корректном управлении приватными ключами.

- ChaCha20-Poly1305 считается современным AEAD-стандартом, эффективным на ARM-платформах (включая Raspberry Pi).

- Отсутствие "настраиваемых слабых шифров" снижает риск ошибочной конфигурации.

4.2 Wi-Fi сегмент (WPA2-PSK)

- Шифрование канала: AES-CCMP.

- Риски: при слабом PSK возможны офлайн-атаки на пароль по захваченному handshake.

- Рекомендация: использовать длинный случайный пароль (не менее 16-20 символов, лучше 24+).

4.3 Криптографическая эффективность

1. Используемые примитивы WireGuard соответствуют современным требованиям практической криптостойкости.

2. ChaCha20-Poly1305 хорошо подходит для ARM-устройств, обеспечивая баланс скорости и безопасности.

3. Риск криптографического компромисса на практике в большей степени зависит от защиты приватных ключей и качества PSK Wi-Fi, чем от выбранных алгоритмов.

4.4 Эксплуатационная эффективность

1. Система запускается после подачи питания без ручной настройки.
2. Диагностика прозрачна через стандартные инструменты Linux (``systemctl``, ``wg``, ``ip``, ``iptables/nft``).
3. Архитектура масштабируема: возможно добавление новых peer-узлов в ``wg0`` на NL.

Реализация (итоговая сводка конфигурации)

NL сервер

- Поднят ``wg0`` с адресом ``10.66.0.1/24``.
- Добавлены peers:
 - RU: ``10.66.0.2/32`` (существующая инфраструктура),
 - RPi: ``10.66.0.3/32`` (новый шлюз).
- Включен ``net.ipv4.ip_forward=1``.
- Настроен NAT для VPN-подсети в ``eth0``.

Raspberry Pi

- Интерфейс AP: ``ap0`` с адресом ``192.168.77.1/24``.
- DHCP диапазон: ``192.168.77.50 - 192.168.77.200``.
- WireGuard интерфейс: ``wg0`` с адресом ``10.66.0.3/24``.
- Политика форвардинга: только ``ap0 -> wg0``.
- Автозапуск сервисов после перезагрузки настроен.

Тестирование и результаты

Проведенные проверки

1. Проверка статуса сервисов (``active``): ``ap0``, ``hostapd``, ``dnsmasq``, ``wg-quick@wg0``.
2. Проверка WireGuard handshake между RPi и NL.
3. Проверка выдачи DHCP-адресов клиентам в подсети ``192.168.77.0/24``.
4. Проверка внешнего IP с Raspberry Pi и клиентов (``api.ipify.org``) — соответствует IP NL-сервера.
5. Проверка сценария cold boot: после отключения/включения питания Wi-Fi сеть поднимается.

Полученные результаты

- Целевая функциональность достигнута: Raspberry Pi работает как автономный Wi-Fi роутер с выходом через NL.
- Трафик клиентов передается через защищенный WireGuard-туннель.

- Система работоспособна в режиме "подключил питание -> получил готовый защищенный Wi-Fi".

4.5 Анализ рисков и меры снижения

1. Компрометация приватных ключей WireGuard
 - Меры: ограничение прав доступа (`chmod 600`), не передавать ключи в открытых каналах.
2. Слабый пароль Wi-Fi
 - Меры: длинный случайный PSK, периодическая ротация.
3. Нарушение порядка старта сервисов после reboot
 - Меры: `ap0.service` + зависимости `Before/After` для `hostapd` и `dnsmasq`.
4. Утечка трафика мимо VPN
 - Меры: правила FORWARD/NAT только на `wg0` для клиентской подсети.
5. Человеческий фактор при ручных изменениях firewall
 - Меры: сохранение правил (`netfilter-persistent`), бэкапы конфигов перед правками.

Практические рекомендации по усилению безопасности

1. Перевести Wi-Fi на WPA3-SAE при поддержке клиентских устройств.
2. Использовать отдельный VLAN/SSID для недоверенных устройств (IoT).
3. Ограничить управление Raspberry Pi по SSH:
 - вход только по ключу;
 - отключить PasswordAuthentication;
 - ограничить по IP.
4. Включить централизованный журнал и мониторинг (например, `journald` + удаленный `syslog`).
5. Регулярно обновлять пакеты безопасности ОС и WireGuard.
6. Выполнять ротацию ключей WireGuard по регламенту.

5 ПРОБЛЕМЫ, ВЫЯВЛЕННЫЕ НА ПРАКТИКЕ, И ИХ РЕШЕНИЕ

5.1 Ошибка запуска dnsmasq: unknown interface ap0

Проблема. `dnsmasq` не стартовал, так как интерфейс `ap0` отсутствовал в момент старта.

Причина. Нарушение порядка инициализации сервисов после загрузки.

Решение.

1. создан отдельный `ap0.service`;
2. добавлены systemd-зависимости `Requires/After` для `hostapd` и `dnsmasq`;
3. перезапущены сервисы в корректном порядке.

Результат. `dnsmasq` стабильно запускается и выдает DHCP-адреса.

5.2 SSID виден, но клиенты периодически не подключались после reboot

Проблема. Точка доступа иногда поднималась, но подключение клиента не завершалось устойчиво.

Причина. Конкуренция служб при старте и неполная инициализация AP-интерфейса.

Решение.

1. сделан idempotent-юнит `ap0.service` (без падения, если интерфейс уже существует);
2. добавлена повторная инициализация адреса `192.168.77.1/24` при старте;
3. зафиксирован порядок старта AP-стека.

Результат. После перезагрузки сеть поднимается в рабочем состоянии.

5.3 Проблемы маршрутизации при первичной отладке туннеля

Проблема. Трафик в отдельных состояниях шел не по ожидаемому пути или зависал.

Причина. Несогласованность `AllowedIPs`, таблиц маршрутизации и правил NAT/FORWARD на ранних этапах отладки.

Решение.

1. выровнен адресный план;
2. нормализованы правила FORWARD/NAT;
3. добавлена верификация через `wg show`, `ip rule`, `ip route`, `curl api.ipify.org`.

Результат. Внешний IP клиента стабильно соответствует NL-серверу.

5.4 Сосуществование с уже работающими сервисами на NL

Проблема. На NL уже были активны контейнеры Amnezia, и существовал риск их деградации при изменении firewall.

Причина. Общий стек netfilter для нескольких подсистем.

Решение.

1. перед изменениями выполнен аудит текущих правил;
2. новые правила добавлялись без удаления рабочих цепочек;
3. применен принцип минимально необходимого вмешательства.

Результат. Новый контур RPi->NL внедрен без нарушения существующих сервисов.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы спроектирован и реализован защищенный сетевой шлюз на базе Raspberry Pi 4 с выводом трафика через удаленный сервер в Нидерландах. Решение обеспечивает конфиденциальность и целостность передаваемых данных за счет использования современных криптографических примитивов WireGuard (Curve25519, ChaCha20-Poly1305) и шифрования беспроводного сегмента WPA2-CCMP.

Поставленная цель достигнута: реализован автономный режим работы, при котором после подачи питания Raspberry Pi автоматически поднимает Wi-Fi сеть и обеспечивает доступ в интернет через зашифрованный VPN-туннель. Результаты подтверждают практическую применимость подхода для персонального защищенного доступа и учебных задач по дисциплине "Криптографические методы обеспечения информационной безопасности".

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Donenfeld J. WireGuard: Next Generation Kernel Network Tunnel. URL: <https://www.wireguard.com/>
2. WireGuard Documentation (Quick Start, Cryptokey Routing). URL: <https://www.wireguard.com/quickstart/>
3. RFC 8439: ChaCha20 and Poly1305 for IETF Protocols. URL: <https://datatracker.ietf.org/doc/rfc8439/>
4. Bernstein D. J. Curve25519: new Diffie-Hellman speed records. URL: <https://cr.yp.to/ecdh.html>
5. hostapd (Linux man page). URL: <https://w1.fi/hostapd/>
6. dnsmasq (Linux man page). URL: <https://thekelleys.org.uk/dnsmasq/doc.html>
7. Netfilter/iptables Documentation. URL: <https://www.netfilter.org/documentation/>
8. Ubuntu Server Documentation (Networking, Systemd, Netplan). URL: <https://ubuntu.com/server/docs>